Manual del Programador — JuegoDeNaves (Motor POO)

Proyecto: `juegodenaves` — Actividad 8 (LPR 5.º 3.ª) **Versión:** 1.0.0 **Autor:** Ian Quiroga **Fecha:** 2026 **Plataforma:** Windows 10/11 (depende de `windows.h` y `conio.h`)

---

1. Descripción del sistema

Simulador de combate espacial en consola construido con **Programación Orientada a Objetos**. El motor define las clases `Nave`, `Asteroide`, `Bala` y `DisparoRapido`, con atributos privados y métodos públicos que controlan el movimiento, las colisiones y el HUD.

Incluye el **Desafío Obligatorio** implementado: el *Sistema de power-ups con Disparo Rápido (Rapid Fire) temporal*.

2. Arquitectura del sistema

```
main()
 ■■■ Pide nombre del piloto (anti-copia: visible en la captura)
 ■■■ system("cls") + ocultarCursor()
 ■■■ Crea: Nave jugador(35, 20, 3)
 ■          DisparoRapido arma(35, 20)
 ■          vector<Asteroide*> asteroides  (5 elementos)
 ■          vector<Bala*>      balas
 ■          vector<DisparoRapido*> powerUps
 ■
 ■■■ while (!gameOver)
      ■■■ HUD: PILOTO / VIDAS / PUNTOS / RAPID FIRE
      ■■■ _kbhit() → ' ' dispara (1 o 3 balas) | W A S D mueve
      ■■■ Mover + purgar balas   (recorrido inverso)
      ■■■ arma.tick()            (descuenta el temporizador)
      ■■■ Mover powerUps
      ■■■ Mover asteroides
      ■    ■■■ Colisión Asteroide → Nave   (resta vida)
      ■    ■■■ Colisión Bala → Asteroide   (+10 pts, 35 % power-up)
      ■■■ Colisión Nave → PowerUp           (+5 pts, activa 8 s)
      ■■■ Sleep(30)
```

3. Especificación de las clases

3.1 `class Nave`

```
class Nave {
private:  int x, y;        // posición en la consola
          int vidas;       // cantidad de impactos tolerados
public:
    Nave(int _x, int _y, int _vidas);  // CONSTRUCTOR
    int  getX();
    int  getY();
    int  getVidas();
    void reducirVida();    // vidas--
    void pintar();         // dibuja " ^ " / " (O) " / "^^^^ "
    void borrar();         // borra el dibujo
    void mover(char tecla);// W/A/S/D con límites de la consola
};
```

Atributo Restricción Quién puede escribirlo

`x`, `y`
`2 ≤ x ≤ 70`, `3 ≤ y ≤ 20`
Solo `mover()`

`vidas`
`≥ 0`
Solo el constructor y `reducirVida()`

Invariante clave: nadie desde `main()` puede asignar `vidas = -5`. Ese es el motivo de que `vidas` sea `private`.

3.2 `class Asteroide`

Atributos `x`, `y` privados. `mover()` incrementa `y`; al superar la fila 23 se **reposiciona** en una columna aleatoria de la fila 4 (nuevo asteroide desde arriba).

3.3 `class Bala`

`mover()` borra la posición anterior y dibuja `|` en `y - 1`. `fueraDeRango()` devuelve `true` cuando `y < 4`, lo que dispara su eliminación.

3.4 `class DisparoRapido` — *Desafío Obligatorio*

```
class DisparoRapido {
private:
    int  x, y;               // posición del icono 'R'
    bool activo;             // ¿está en pantalla?
    int  segundosRestantes;  // TEMPORIZADOR (privado)
    static const int DURACION = 8;
public:
    DisparoRapido(int _x, int _y);  // CONSTRUCTOR
    bool estaEnPantalla();
    bool estaActivo();      // segundosRestantes > 0
    int  getSegundos();
    void activar();         // ← única vía para iniciar el efecto
    void tick();            // descuenta 1 segundo
    void pintar();          // dibuja 'R'
    void borrar();
    void mover();
};
```

Invariante: `segundosRestantes` nunca es negativo ni mayor que `DURACION`, y `main()` **jamás** lo escribe directamente. Solo puede llamar a `activar()`.

4. Detección de colisiones

Colisión Prueba Efecto
Asteroide → Nave `x ∈ [nave.x, nave.x+4]` **y** `y ∈ [nave.y, nave.y+2]` `-1` vida, reposiciona el asteroide
Bala → Asteroide `x` igual **y** (`y` igual **o** `y+1` igual) `+10` puntos, 35 % de power-up

Colisión Prueba Efecto
Nave → PowerUp Mismo criterio de caja que Asteroide → Nave +5 puntos, activa 8 s

La comparación se hace por rangos y no por igualdad exacta porque los gráficos son de **varias celdas de ancho**: un asteroide colisiona si *cualquiera* de sus celdas toca el recuadro de la nave.

5. Recorrido inverso y borrado de elementos

Los tres vectores se recorren **de atrás hacia adelante** (`size() - 1` hasta `0`):

```
for (int i = (int)balas.size() - 1; i >= 0; i--) { ... erase(balas.begin() + i); }
```

Al `erase`, todos los elementos posteriores se desplazan una posición a la izquierda. Si se recorriera de adelante hacia atrás, esos elementos se **saltarían** durante esa vuelta. El recorrido inverso garantiza que no se pierda ningún elemento.

6. Gestión de memoria

Todos los objetos se crean con `new` y se destruyen explícitamente:

Momento Acción
Bala fuera de rango `delete balas[i];` + `erase`
Bala que impacta `delete balas[j];` + `erase`
Power-up recolectado `delete powerUps[k];` + `erase`
Fin del programa `delete` de todos los asteroides, balas y power-ups restantes

Sin ese `delete` final, los objetos que quedaron en los vectores al terminar la partida quedarían **asignados sin liberar** (*memory leak*).

7. Compilación

```
g++ src/main.cpp -o src/juegodenaves.exe
```

> ■ **Solo Windows.** Usa `windows.h` (`GetStdHandle`, `SetConsoleCursorPosition`) > y `conio.h` (`_kbhit`, `_getch`). En Linux o macOS no compila: habría que > reemplazar esas llamadas por `ncurses` o termios.

8. Errores conocidos y soluciones

Síntoma Causa Solución
`'_kbhit': was not declared` Falta `#include <conio.h>` o no es Windows Verificar que la plataforma sea Windows
La nave desaparece `mover()` no aplica límites en versiones previas Usar la versión con `if (tecla=='a' && x > 2)`
El HUD deja restos de texto No se limpian los espacios del `cout` Los `cout` del HUD ya imprimen espacios de relleno
Las balas «saltean» alguna `erase` en recorrido directo Usar el recorrido inverso
El poder no se activa `activar()` no está siendo invocado Verificar la colisión Nave → PowerUp

9. Extensiones posibles

• **Más funcionalidades del anexo:** escudo de vidas, invocación de aliados,

olas de enemigos, ranking, pausa, turbo, minas.

• **Sonido:** `Beep()` de `windows.h` en cada disparo.

• **Oleadas progresivas:** 1 Clonar la fila de asteroides.

• **Multijugador local:** dos `Nave` con teclas distintas.

• **SDL/SFML:** migrar el motor a una ventana gráfica sin reescribir las clases.

10. Control de versiones

Versión Estado Descripción
`v1.0.0` Baseline docente Motor base: clases `Nave`, `Asteroide`, `Bala`, colisiones y HUD.
`v1.1.0` **Baseline aprobado** Desafío obligatorio: `DisparoRapido`, power-ups (35 %), disparos triples de 8 s, marcador de puntos y limpieza de memoria.